

CLUB SMPE — SAINT-MALO PLONGÉE EMERAUDE

Notice technique — diveSMPE

Structure de l'application, rôle de chaque script, module et widget, et notice d'utilisation pour smartphone, tablette et ordinateur.



diveSMPE

Baie de Saint-Malo · Application PWA offline

Notice technique & notice d'utilisation

01/09/2026

Sommaire

Partie A — Notice d'utilisation	5
A.1 Ce que fait l'application	5
A.2 Sur quel appareil ? Installation	5
A.3 Se connecter (lien par email)	6
A.4 L'écran principal	6
A.5 Trouver un site de plongée	7
A.6 La fiche d'un site	8
A.7 Navigation GPS	8
A.8 Marées	9
A.9 Prédiction de plongeabilité	9
A.10 Planificateur bi-journée (2 plongées)	9
A.11 Courants de marée	10
A.12 Port — créneaux de sortie	10
A.13 Météo marine	10
A.14 Retour d'expérience post-plongée	11
A.15 Exporter les marées en CSV	11
A.16 Mode hors-ligne et mise à jour	11
A.17 Questions fréquentes	12
Partie B — Architecture technique	13
B.1 Les deux mondes	13
B.2 Schéma d'ensemble	13
B.3 Arborescence du dépôt	14
B.4 Pipeline de preprocessing (<code>r/</code>)	14
<code>build_all.R</code> — script maître	14
<code>02_process_bdd.R</code> — sites → GeoJSON	15
<code>01_process_las.R</code> — LiDAR → bathymétrie	15
<code>04_marees_fes.py</code> — marées FES2022	15
<code>05_courants_fes.py</code> — courants de marée (hors build)	16
<code>03_generate_profile.R</code> — utilitaire ponctuel	16
B.5 La PWA — structure et chargement	16
Ordre de chargement des scripts (<code>index.html</code>)	16
Séquence d'initialisation (<code>app.js</code> → <code>_startApp</code>)	16
Dépendances entre modules	17
B.6 Modules JavaScript, un par un	17
<code>config.js</code> — configuration globale (<code>CONFIG</code>)	17
<code>app.js</code> — orchestrateur (<code>App</code>)	18
<code>auth.js</code> — authentification (<code>Auth</code>)	18
<code>carte.js</code> — carte Leaflet (<code>Carte</code>)	18
<code>sites.js</code> — sites : liste + fiche (<code>Sites</code>)	19
<code>marees.js</code> — marées (<code>Marees</code>)	19
<code>mareesite.js</code> — moteur de plongeabilité (<code>MaréeSite</code>)	19

<code>bathy.js</code> — bathymétrie LiDAR (<code>Bathy</code>)	19
<code>navigation.js</code> — GPS & cap (<code>Navigation</code>)	20
<code>meteo.js</code> — météo marine (<code>Meteo</code>)	20
<code>courants.js</code> — courants de marée (<code>Courants</code>)	20
<code>port.js</code> — seuil du port (<code>Port</code>)	20
<code>prevision.js</code> — prévision de plongeabilité (<code>Prevision</code>)	20
<code>biplongee.js</code> — planificateur bi-journée (<code>BiPlongee</code>)	21
<code>retourexperience.js</code> — retour d'expérience (<code>RetourExperience</code>)	21
<code>mareesexport.js</code> — export CSV (<code>MareesExport</code>)	21
<code>cgu.js</code> — modale CGU (<code>Cgu</code>)	21
<code>tutorial.js</code> — tutoriel guidé (<code>Tutorial</code>)	21
B.7 Widgets et éléments d'interface	21
B.8 Service Worker et cache hors-ligne	22
B.9 Le Worker Cloudflare	22
B.10 Les scripts Google Apps Script	23
B.11 Authentification	23
B.12 Formats de données	24
<code>sites.geojson</code> / réponse GET <code>/sites</code>	24
<code>marees.json</code> / réponse GET <code>/marees</code>	25
<code>bathy_sites.json</code>	25
<code>courants_grid.json</code>	25
Miniatures	25
B.13 Les codes marée	26
B.14 Calibration des marées FES2022	26
B.15 Déploiement	26
PWA → GitHub Pages	26
Worker Cloudflare	27
Contrainte HTTPS	27
Partie C — Maintenance & exploitation	28
C.1 Cycle de mise à jour complet	28
C.2 Ajouter ou modifier un site	28
C.3 Régénérer les données	28
C.4 Publier une nouvelle version de la PWA	28
C.5 Gérer les utilisateurs	29
C.6 Prolonger la période des marées	29
C.7 Secrets et variables d'environnement	29
C.8 Dépannage	30
Annexes	31
Glossaire	31
Documents de conception (<code>specs/</code>)	31
Autres documents du dépôt	32
Sources de données et attributions	32

Application de catalogue et d'aide à la navigation pour les sites de plongée du club SMPE (Saint-Malo Plongée Emeraude) — Baie de Saint-Malo.

Ce document décrit :

- **Partie A — Notice d'utilisation** : comment se servir de l'application, pour tous les membres du club (aucune connaissance technique requise).
- **Partie B — Architecture technique** : structure du projet, rôle de chaque script / module / widget, formats de données, pour les personnes qui maintiennent l'outil.
- **Partie C — Maintenance & exploitation** : cycle de mise à jour, ajout d'un site, d'un utilisateur, dépannage.

i Information

Version de la notice : 2026-09-01. Elle décrit le Service Worker v59 , l'authentification par lien magique et les données servies « en direct » par le Worker Cloudflare.

Partie A — Notice d'utilisation

A.1 Ce que fait l'application

diveSMPE réunit, sur une seule carte marine utilisable **sans réseau**, tout ce qui aide à choisir un site de plongée dans la Baie de Saint-Malo et à s'y rendre :

- Carte interactive avec les ~60 sites du club (récifs, épaves, roches).
- Fiche par site : type, niveau, accès, mouillage, conditions, commentaire du responsable technique.
- **Marées** en temps réel : hauteur d'eau actuelle, PM/BM, coefficient, courbe.
- **Fenêtre de plongée optimale** par site (calculée à partir des codes de marée du site et de l'étale du jour).
- **Bathymétrie LiDAR** : vue du fond, profil bathymétrique, transect libre.
- **Courants de marée** animés sur la carte (modèle harmonique).
- **Port** : le passage de la cale du Naye est-il ouvert, maintenant et sur la journée, pour chaque bateau du club ?
- **Prévision** : simuler une date/heure future et voir les sites plongeables.
- **Planificateur bi-journée** : combinaisons de 2 plongées réalisables.
- **Navigation GPS** : cap, distance, ETA vers le site sélectionné.
- **Météo marine** (si réseau) : vent, houle, visibilité.
- **Retour d'expérience** : formulaire post-plongée renvoyé au club.

⚠ Attention

diveSMPE est un **outil d'aide à la décision**. Il ne remplace ni le jugement du directeur de plongée, ni les sources officielles (SHOM, Météo-France, préfecture maritime), ni un équipement de navigation homologué. Voir les CGU (bouton  →  CGU).

A.2 Sur quel appareil ? Installation

diveSMPE est une **application web** : elle s'ouvre dans un navigateur, sur n'importe quel appareil, sans rien installer depuis un store.

Appareil	Usage typique	Remarques
Smartphone (Android / iPhone)	En mer, dans la poche : marées, fenêtres de plongée, GPS, navigation vers le site	GPS et boussole intégrés ; installation « écran d'accueil » recommandée
Tablette (Android / iPad)	À bord, en consultation partagée : carte, bathymétrie, prévision	Grand écran ; la fiche site s'affiche sur le côté en mode paysage
Ordinateur (Windows / Mac / Linux)	Au centre, en préparation de sortie : prévision, bi-journée, export marées, retour d'expérience	Pas de GPS en général ; interface adaptée au clavier/souris

Navigateurs : **Chrome** ou **Edge** (recommandés, Android comme ordinateur), **Safari** (iPhone / iPad / Mac), **Firefox**. Il faut un navigateur récent (2021 ou plus) pour le fonctionnement hors-ligne.

Installer l'application (facultatif mais conseillé sur mobile) :

- **Android / ordinateur (Chrome, Edge)** : ouvrir <https://gallonr.github.io/diveSMPE/>, puis menu du navigateur → « Installer l'application » / « Ajouter à l'écran d'accueil ».
- **iPhone / iPad (Safari)** : ouvrir l'adresse, bouton Partager → « Sur l'écran d'accueil ».

- Une fois installée, l'application se lance en plein écran, avec son icône, comme une application native — et **sans réseau**.

Première visite : elle doit se faire **en ligne** (WiFi du centre ou données mobiles). L'application se met alors en cache et fonctionne ensuite hors-ligne (voir A.16). Chaque appareil se connecte une fois avec son propre lien (voir A.3).

A.3 Se connecter (lien par email)

L'accès est **réservé aux membres autorisés** par la direction du club. Chaque personne a un compte individuel identifié par son adresse email. La connexion se fait **appareil par appareil** : un même membre peut connecter son smartphone, sa tablette et son ordinateur, chacun avec sa propre demande de lien.

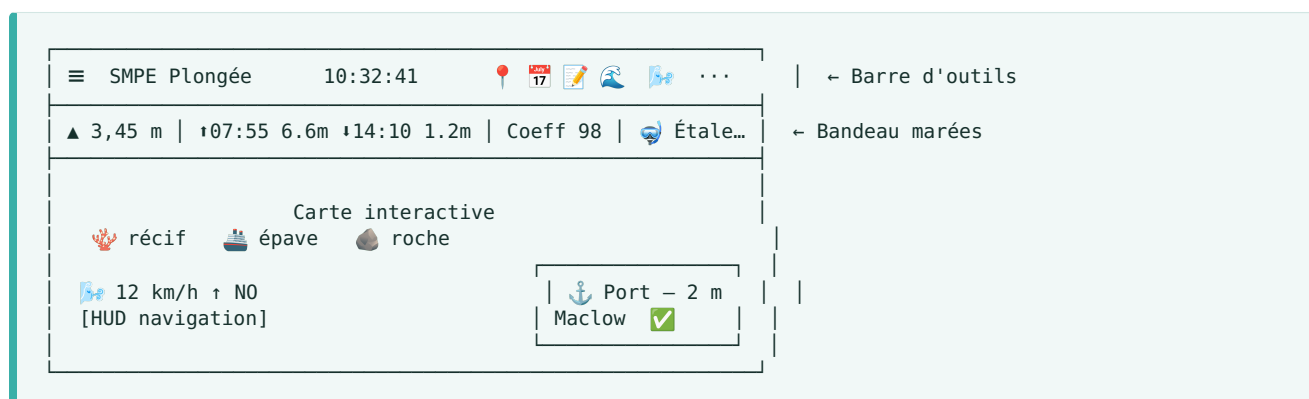
1. À l'ouverture, l'écran de connexion demande votre **email**.
2. Saisir l'email connu du club, puis « Recevoir mon lien ».
3. Vous recevez un email « Votre lien de connexion — SMPE Plongée ». Deux possibilités :
 - **Ouvrir le lien** depuis l'appareil concerné : l'application s'ouvre et vous connecte automatiquement.
 - Si le lien s'ouvre dans le mauvais navigateur (ou une autre application) : copier le **code (token)** figurant dans l'email et le coller dans le champ « Ou collez le token reçu par email », puis « Valider le token ».
4. Une fois connecté, la session reste valable **90 jours**, y compris hors ligne. Elle se prolonge automatiquement à chaque utilisation en ligne.

Points importants :

- Le lien est lié à **l'appareil** qui a fait la demande : un token copié sur un autre appareil est refusé. Faites la demande depuis l'appareil que vous utiliserez pour plonger, et refaites-en une sur chaque autre appareil.
- « Cet email n'est pas reconnu » → l'adresse n'est pas (ou plus) dans la liste des membres : contacter le club.
- « Votre accès a été révoqué » → le compte a été désactivé par le club.
- Se déconnecter : bouton  →  Déconnexion.

A.4 L'écran principal

L'interface est **responsive** : sur mobile, la liste des sites et la fiche s'ouvrent en panneaux plein écran ; sur tablette en paysage et sur ordinateur, la fiche s'affiche sur le côté et la carte reste visible. Les fonctions sont les mêmes partout.



Barre d'outils (en haut)

Bouton	Fonction
	Ouvre/ferme la liste des sites (panneau de gauche)

Bouton	Fonction
horloge	Heure locale de l'appareil (mise à jour chaque seconde)
	Centre la carte sur votre position GPS
	Prévision de plongeabilité + planificateur bi-journée
	Retour d'expérience post-plongée (pastille = envois en attente)
	Fenêtre marées (courbe J-1 → J+2, tableau PM/BM)
	Météo marine (nécessite le réseau)
...	Documentation, tutoriel, export marées, CGU, déconnexion

Bandeau marées (toujours visible) — affiche en permanence :

- la **hauteur d'eau actuelle** en mètres, avec ▲ (montante) ou ▼ (descendante) ;
- les **PM/BM du jour** triées par heure (↑ pleine mer, ↓ basse mer) ;
- le **coefficient** du jour ;
- «  Étale PM dans ...min » quand une étale de pleine mer approche (≤ 2 h).

Sur la carte

- Un **onglet «  Widgets »** (bord gauche) masque/affiche d'un coup le HUD de navigation, le widget port et le widget vent.
- Le sélecteur de couches (coin haut-droit) permet de changer de fond de carte et d'activer des overlays (voir A.11 et B.7).
- Le bouton  (coin haut-droit) filtre les sites par type de mouillage.

Tutoriel — au tout premier lancement, un tutoriel guidé présente les fonctions principales. On peut le relancer à tout moment via  →  Tutoriel.

A.5 Trouver un site de plongée

Depuis la carte — chaque site est un marqueur coloré selon son type ( récif,  épave,  roche). Une **pastille** sur le marqueur indique l'état de plongée du moment :

Pastille	Signification
 vert	Fenêtre de plongée ouverte maintenant
 orange	Fenêtre dans moins de 2 h
 rouge	Fenêtre passée, ou conditions de marée non favorables
(aucune)	Pas d'information de marée pour ce site

Toucher un marqueur → petite bulle → «  Voir la fiche ».

Depuis la liste (bouton ≡) :

- **Recherche** par nom de site.
- **Filtre par type** : Tous / Récif / Épave / Roche.

- **Filtre par profondeur** : ≤ 6 m / ≤ 10 m / ≤ 20 m / +20 m — calculé avec la **hauteur de marée actuelle** (profondeur réelle, pas la cote « zéro »).
- **Filtre par mouillage** (widget  sur la carte) : Fixe  / Ancre  / Gueuse .

Les filtres se cumulent, et s'appliquent aussi aux marqueurs de la carte. Chaque ligne de la liste affiche un badge de couleur identique à la pastille des marqueurs.

A.6 La fiche d'un site

La fiche s'ouvre en bas de l'écran (mobile) ou sur le côté (tablette en paysage). Elle a **trois onglets**.

Onglet Infos

- Type de plongée, niveau requis, accessibilité, mouillage.
- **Bloc marée interprété** : badge d'état (//) , code de marée brut du site, phrase en clair (« Pleine Mer vive-eau à l'étable », ...), prochaine fenêtre de plongée avec ses horaires, durée de la fenêtre.
- **Bloc courants** (si la couche courants est disponible) : vitesse en nœuds et direction du courant au point le plus proche, pour l'instant affiché sur la carte.
- **Profondeur actuelle** : profondeur mini/maxi du site **corrigée de la marée du moment** (cote LiDAR + hauteur d'eau). « Données bathymétriques LiDAR à intégrer » si le site n'est pas couvert par le LiDAR.
- **Horaires de plongée** : deux champs (début / fin) pré-remplis avec la fenêtre calculée ; la durée s'affiche automatiquement. Modifiables à la main.
- Commentaire du responsable technique.

Onglet Bathymétrie

- **Miniature MNT** : vue de dessus du fond, colorée selon la profondeur (issue du LiDAR LITTO3D, résolution 0,5 m).
- **Profil bathymétrique** : coupe Est → Ouest du fond, avec la ligne de surface positionnée à la hauteur de marée actuelle.
- **Tracer un transect libre** : bouton  , puis cliquer un point **A** et un point **B** sur la miniature → le profil se recalcule le long de cet axe.  **Défaut** revient au transect Est → Ouest.
- **Overlay carte** : le curseur d'opacité superpose la miniature du fond directement sur la carte.

Onglet Conditions

- Accès vent (orientations à éviter), sensibilité à la houle.
- **Météo temps réel** au point du site (vent, température, code météo) — si réseau. « Météo non disponible offline » sinon.

Depuis la fiche : bouton  **Naviguer vers ce site** (voir A.7) et bouton  **Retour d'expérience** pré-renseigné avec ce site.

A.7 Navigation GPS

i Information

Nécessite un appareil équipé d'un GPS (smartphones et la plupart des tablettes ; rarement les ordinateurs de bureau). Sur un appareil sans GPS, la carte, les fiches, les marées et la prévision restent pleinement utilisables ; seule la position en direct et la navigation vers le site sont indisponibles.

-  dans la barre d'outils : la carte se centre sur votre position, un marqueur pulsant apparaît. Le GPS est démarré automatiquement au lancement de l'application (si vous en autorisez l'accès).

- Le **HUD de navigation** (coin bas-gauche) affiche :
 - vos coordonnées en **degrés / minutes / secondes** (N/S, E/O) ;
 - le **cap** (fourni par le GPS quand vous êtes en mouvement) ;
 - la **vitesse** en nœuds (en rouge au-delà de 10 kt) ;
 - la **distance** au site sélectionné en milles nautiques.
- Depuis une fiche site,  **Naviguer vers ce site** : une ligne verte pointillée relie votre position au site ; le bloc affiche distance, cap et **ETA** (à 6 nœuds par défaut). Bouton  **Arrêter la navigation** dans le HUD.

A.8 Marées

Bouton .

- **Courbe** sur 4 jours (J-1 → J+2), trait vertical « maintenant ».
- **Tableau PM/BM** : heure, hauteur, coefficient.
- Les hauteurs sont exprimées en **mètres au-dessus du zéro hydrographique (ZH SHOM Saint-Malo)**, calculées à partir du modèle **FES2022** (CNES/LEGOS) puis **recalées sur l'annuaire SHOM** (voir B.14).
- La hauteur d'eau à un instant donné est obtenue par **interpolation sinusoïdale** entre les extrêmes (règle des douzièmes simplifiée).

Précision indicative : ± 20 cm sur la hauteur, ± 5 min sur l'heure des étales. Pour une plongée technique, se référer à l'annuaire officiel SHOM.

A.9 Prévision de plongabilité

Bouton .

1. Choisir une **date** et une **heure**.
2. « Calculer ».
3. L'application affiche :
 - la **hauteur de marée** prévue et le coefficient / type d'eau ;
 - une **mini-courbe** de la journée avec un repère à l'heure choisie ;
 - l'**état du port** pour chaque bateau à cette heure ;
 - la **liste des sites** classés :  plongeables maintenant →  bientôt →  non plongeables → — sans contrainte de marée. Chaque carte indique la fenêtre, la profondeur ZH, la profondeur réelle estimée et le courant.
4. Filtre **profondeur max réelle** : Toutes / ≤ 6 m / ≤ 10 m / ≤ 20 m / +20 m.

Si le club a renseigné une colonne « sites prioritaires », seuls ces sites sont affichés par défaut, avec un bouton « Voir tous les sites ».

i Information

La prévision ne fonctionne que sur la plage couverte par les données de marées (aujourd'hui → fin 2028, voir C.6).

A.10 Planificateur bi-journée (2 plongées)

Dans la fenêtre Prévision, cocher « **2 tanks** ».

L'heure saisie devient l'**heure de départ de la cale du Naye**. On règle l'**intervalle de surface** autorisé (min / max, défaut 30 → 180 min). « Calculer » examine toutes les combinaisons (**site 1, site 2**) et ne garde que celles où :

- les **deux plongées** tombent dans une fenêtre d'étale du site concerné ;
- l'intervalle de surface (transit inclus) reste dans les bornes réglées ;
- le **profil n'est pas inversé** : la 2^e plongée n'est pas plus profonde que la 1^{re} (tolérance 5 m si la profondeur réelle ≤ 20 m, sinon 0 m).

Hypothèses de calcul : bateau **Maclow à 15 nœuds**, distances à vol d'oiseau multipliées par **1,35** (contournement des hauts-fonds / chenaux), **45 min** par plongée. Chaque résultat indique la **fenêtre de départ** possible, les horaires des deux plongées, les profondeurs, les transits et l'**heure de retour au port** (avec alerte si le seuil est fermé à ce moment). Un champ de recherche filtre les résultats par nom de site.

A.11 Courants de marée

Dans le sélecteur de couches (coin haut-droit), activer «  **Courants marée** ».

- Des **flèches colorées** apparaissent : la flèche pointe **vers où va** le courant (convention nautique), la couleur donne l'intensité (bleu < 0,25 kn · émeraude ~0,5 kn · orange ~1 kn · rouge > 1,5 kn), la longueur est proportionnelle à la vitesse.
- Le petit panneau (coin bas-droit) permet :
 -  : revenir à « maintenant » (rafraîchi toutes les 5 min) ;
 -  /  : animer les courants heure par heure sur 24 h ;
 - un sélecteur **date/heure** pour une échéance précise.
- Quand un site est sélectionné, sa fiche (onglet Infos) affiche le courant au point de grille le plus proche.

⚠ Attention

Le modèle (FES) a une maille d'environ **3,7 km** : les effets locaux (chenaux, caps, récifs) ne sont **pas** résolus et peuvent différer fortement. La couche « Courants » n'apparaît que si les données ont été générées au centre (voir C.3).

A.12 Port — créneaux de sortie

Un **widget flottant** (coin bas-droit) indique en permanence, pour chaque bateau du club, si le passage du **seuil de la cale du Naye** est possible :

- Condition : $\text{hauteur de marée (ZH)} \geq \text{seuil (2,0 m)} + \text{tirant d'eau du bateau}$.
- Bateaux configurés : **Maclow** (1,3 m), **Cassiopée** (1,1 m), **Neptune** (0,7 m).
- Le widget affiche  /  par bateau, la **prochaine heure d'accès** si bloqué, ou la **prochaine heure de blocage** si actuellement ouvert.

Dans la fenêtre Prévision, le bloc  **Port** détaille les **plages horaires de blocage** de chaque bateau pour la journée choisie.

A.13 Météo marine

Bouton  (nécessite le réseau 4G ou WiFi).

Données **Météo-France (modèle AROME) via Open-Meteo**, sans clé :

- Conditions actuelles : température, ciel (code météo), vent + rafales, visibilité.
- **Houle** : hauteur significative, période, houle de fond (swell).

- **Prévision du vent** heure par heure sur 24 h.

Les données sont mises en cache 30 min. En l'absence de réseau : « Pas de connexion réseau ».

A.14 Retour d'expérience post-plongée

Bouton  (barre d'outils) ou depuis une fiche site.

Formulaire : site et date (**obligatoires**), heures de mise à l'eau / sortie, bateau, état de la mer, vent, courant ressenti, commentaire libre. L'application calcule et enregistre automatiquement la hauteur d'eau, l'étale la plus proche et le coefficient du jour.

- Envoyé **avec réseau** : enregistré immédiatement dans le tableur du club.
- **Sans réseau** : mis en **file d'attente locale** ; la pastille sur  indique le nombre d'envois en attente ; ils partent automatiquement au retour du réseau.

A.15 Exporter les marées en CSV

 →  **Exporter marées**. Choisir une période (dans la plage disponible). Le fichier CSV (séparateur **;**, encodage UTF-8) liste, pour chaque PM/BM : date, heure, type, coefficient, hauteur, ainsi que les **créneaux de blocage** des 3 bateaux du club pour la journée.

i Information

Le téléchargement de fichier est fiable dans un navigateur d'ordinateur ; sur mobile (surtout depuis l'application installée), il peut être bloqué par le système. L'export est donc surtout prévu pour un usage sur ordinateur au centre.

A.16 Mode hors-ligne et mise à jour

Dès la première ouverture en ligne, un **Service Worker** met en cache l'application et ses données. Ensuite :

Fonction	Hors-ligne ?
Carte (fonds déjà consultés), sites, marées, bathymétrie, courants	✓ Oui
Fonds de carte non encore consultés, tuiles OpenSeaMap/IGN	⚠ Seulement si pré-chargés en WiFi
Météo, overlays météo (vent, température, précipitations)	✗ Non (réseau requis)
Connexion (demande de lien)	✗ Non — mais la session déjà ouverte reste valable 90 j

Un **bandeau orange** «  **Mode hors-ligne — données locales** » s'affiche quand le réseau est absent.

Mettre à jour avant une sortie (à faire sur chaque appareil emporté) :

1. Connecter l'appareil à Internet (**WiFi du centre** ou données mobiles).
2. Ouvrir l'application ; laisser charger quelques secondes.
3. Si une nouvelle version est disponible, l'application **se recharge automatiquement** pour l'appliquer.
4. Parcourir la zone de plongée sur la carte pour **pré-charger les tuiles** qui seront utiles en mer.

Le cache est **propre à chaque appareil et à chaque navigateur** : préparer sa sortie sur l'ordinateur du centre ne met pas à jour le smartphone qui ira en mer.

Les **métadonnées des sites** et les **marées** sont rechargées en direct depuis le serveur à chaque passage au premier plan / retour du réseau (voir B.9) : une modification faite au centre apparaît sans réinstaller l'application.

A.17 Questions fréquentes

Les profondeurs affichées sont-elles exactes ? Elles combinent la cote du fond (LiDAR LITTO3D, résolution 0,5 m) et la hauteur de marée (FES2022 recalé SHOM). Tolérance : ± 20 cm de marée, $\pm 0,5$ m de fond.

Certains sites n'ont pas de miniature bathymétrique. Le LiDAR disponible ne couvre pas toute la baie (44 sites couverts sur ~60). Ces sites restent utilisables, sans l'onglet Bathymétrie complet.

L'heure de l'étale ne colle pas exactement avec le SHOM. FES2022 a une précision de l'ordre de ± 5 min sur les horaires. Se référer à l'annuaire SHOM pour les plongées techniques.

La carte ne charge plus certaines tuiles. Normal hors-ligne : seules les tuiles déjà consultées sont en cache. Pré-charger la zone en WiFi.

Je ne reçois pas le lien de connexion. Vérifier les spams. Vérifier que l'email est exactement celui déclaré au club. En dernier recours, demander au club de vérifier la ligne dans la liste des membres.

Quel appareil emporter en mer ? Un **smartphone** installé (écran d'accueil) et connecté au moins une fois à jour : il a le GPS et suffit pour les marées, les fenêtres de plongée et la navigation. La tablette est un plus pour la carte et la bathymétrie. L'ordinateur sert surtout à préparer la sortie au centre. Chaque appareil doit être connecté **une fois** et mis à jour **avant** de perdre le réseau.

J'ai changé de téléphone / vidé le cache du navigateur. Le nouvel appareil (ou navigateur) repart de zéro : refaire une demande de lien et rouvrir l'application en ligne pour recharger le cache hors-ligne.

Partie B — Architecture technique

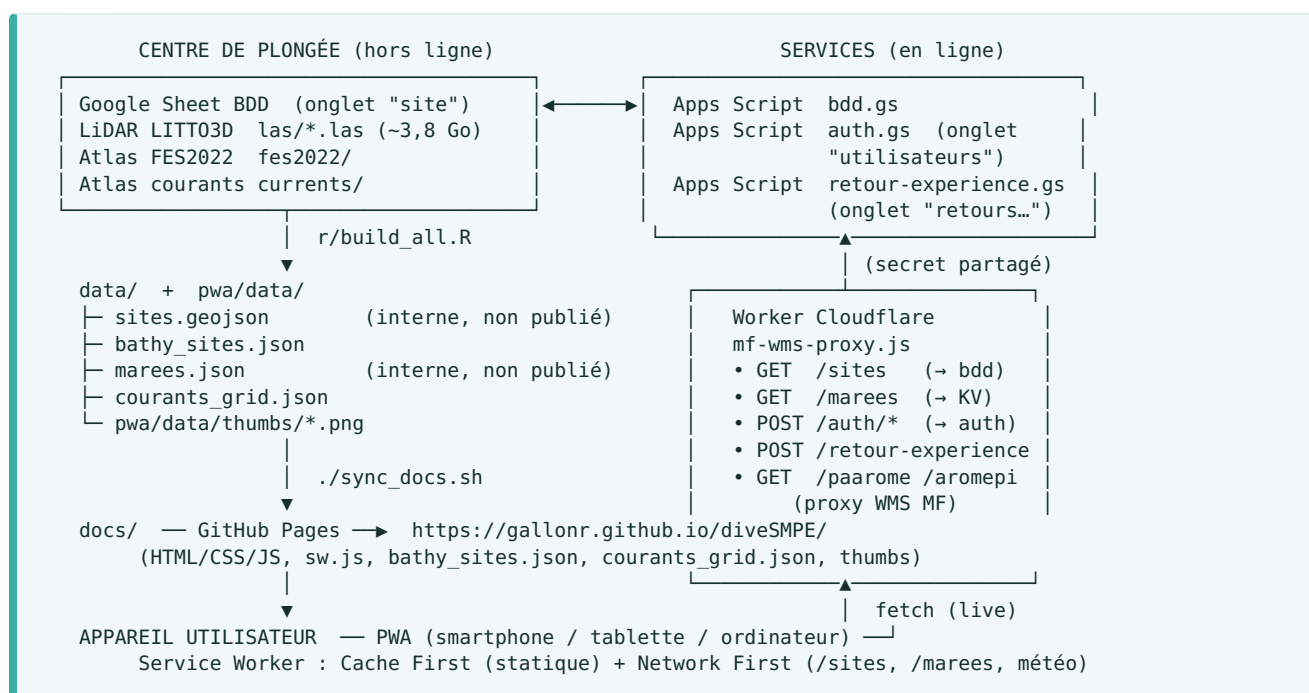
B.1 Les deux mondes

diveSMPE se compose de **deux ensembles indépendants** :

1. **Au centre de plongée (PC)** — un pipeline **R / Python** transforme la base de données des sites et les données brutes (LiDAR, atlas de marées et de courants) en fichiers exploitables par l'application.
2. **Sur l'appareil de l'utilisateur** (smartphone, tablette ou ordinateur, typiquement en mer) — une **PWA** (Progressive Web App) en **Vanilla JS**, sans bundler ni framework, consomme ces fichiers et fonctionne **100 % offline** grâce à un Service Worker. L'interface est responsive et identique sur tous les formats d'écran.

Entre les deux, un **Worker Cloudflare** sert « en direct » les données qui ne doivent pas être publiées dans le dépôt public (base sites, prédictions de marées) et relaie les appels sensibles (Météo-France, authentification, retours d'expérience) vers **Google Apps Script**, qui lit/écrit un **Google Sheet**.

B.2 Schéma d'ensemble



B.3 Arborescence du dépôt

```
CatalogueSitePlongée/
├── r/                               Pipeline de preprocessing
│   ├── build_all.R                 Script maître (enchaîne tout + validation)
│   ├── 02_process_bdd.R           Google Sheet "site" → data/sites.geojson
│   ├── 01_process_las.R           LiDAR LITTO3D → bathy_sites.json + miniatures PNG
│   ├── 03_generate_profile.R      Utilitaire : profil topo depuis un GeoTIFF (hors build)
│   ├── 04_marees_fes.py           Atlas FES2022 → marees.json
│   ├── 05_courants_fes.py         Atlas courants FES → courants_grid.json (hors build)
│   ├── fix_bounds_wgs84.R         Utilitaire ponctuel
│   ├── config_local.R             GOOGLE_SHEET_BDD_ID (gitignoré)
│   └── config_local.R.example
├── bdd/                            README : la BDD sites vit dans un Google Sheet
├── data/                           Sorties du pipeline (dont artefacts non publiés)
├── pwa/                             Application web (source)
│   ├── index.html                 Page unique (SPA)
│   ├── manifest.json             Manifeste PWA
│   ├── sw.js                     Service Worker (constante VERSION)
│   ├── css/style.css
│   ├── js/                       Modules (config, app, auth, carte, sites, marees, ...)
│   ├── data/                     Copie des données + thumbs/ (miniatures MNT)
│   └── libs/                     Leaflet + Turf (copies locales, pour l'offline)
├── docs/                           Copie publiée sur GitHub Pages – NE PAS ÉDITER
│   ├── cloudflare-worker/
│   │   ├── mf-wms-proxy.js        Le Worker (toutes les routes)
│   │   └── wrangler.toml          Nom du Worker + binding KV MAREES_KV
│   └── google-apps-script/
│       ├── bdd.gs                Lecture onglet "site"
│       ├── auth.gs               Lien magique (onglet "utilisateurs")
│       └── retour-experience.gs   Écriture onglet "retours_plongee"
├── specs/                          Notes de conception (design + plans d'implémentation)
├── sync_docs.sh                   pwa/ → docs/ + publication KV + commit/push
├── CLAUDE.md                     Instructions projet (pour Claude Code)
└── NOTICE_TECHNIQUE.md          Ce document
```

Fichiers volumineux / sensibles jamais commités (cf. `.gitignore`) : `las/` (~3,8 Go), `fes2022/`, `currents/`, `data/tiles/`, `data/*.tif*`, `pwa/js/secrets.js`, `pwa/js/tokens.js` (+ équivalents `docs/js/`), `r/`, `config_local.R`, et — depuis la migration du 2026-08-31 — `data/sites.geojson`, `pwa/data/sites.geojson`, `docs/data/sites.geojson`, `data/marees.json`, `pwa/data/marees.json`, `docs/data/marees.json` (servis « en direct » par le Worker).

B.4 Pipeline de preprocessing (`r/`)

`build_all.R` — script maître

Lancé depuis la racine : `Rscript r/build_all.R` (ou `source("r/build_all.R")`). Il enchaîne, avec logs et chronométrage à chaque étape, et **échoue explicitement** si un résultat est manquant :

Étape	Action
1	<code>source("r/02_process_bdd.R")</code> — Google Sheet → <code>data/sites.geojson</code>
1 bis	Copie <code>data/sites.geojson</code> → <code>pwa/data/sites.geojson</code> (doit être frais avant la fusion LiDAR)
2	<code>source("r/01_process_las.R")</code> — LiDAR → <code>bathy_sites.json</code> + miniatures, et fusion <code>profMin / profMax</code> dans <code>pwa/data/sites.geojson</code>
3	<code>python r/04_marees_fes.py</code> — FES2022 → <code>marees.json</code>
4	Copie <code>data/marees.json</code> → <code>pwa/data/marees.json</code> (les autres fichiers sont déjà écrits par l'étape 2)

Étape	Action
5	Validation end-to-end : présence des fichiers, ≥ 50 features GeoJSON, ≥ 360 jours de marées, ≥ 40 sites bathy

i Information

05_courants_fes.py et 03_generate_profile.R **ne sont pas** dans build_all.R : ils se lancent à la main quand c'est nécessaire.

02_process_bdd.R — sites → GeoJSON

- Lit l'onglet `site` du Google Sheet (`googlesheets4::read_sheet`, identifiant `GOOGLE_SHEET_BDD_ID` dans `r/config_local.R`). Authentification `gs4_auth()` : navigateur au 1^{er} lancement, token mis en cache ensuite.
- Exclut les lignes sans `latitude` / `longitude`.
- Convertit en objet `sf`, **CRS WGS84 (EPSG:4326)**, contrôle de l'emprise (`longitude` $\in [-3 ; -1,5]$, `latitude` $\in [48 ; 49,5]$).
- Ne garde que les colonnes utiles (voir B.12), signale en `warning` toute colonne manquante ou valeur `mouillage` hors vocabulaire (`fixe` / `ancre` / `gueuse` / `vide`).
- Écrit `data/sites.geojson` (`COORDINATE_PRECISION=6`, ≈ 11 cm).

i Information

Ce fichier **n'est plus publié**. Il sert d'entrée à `01_process_las.R` et de miroir de contrôle. En production, la PWA récupère ces mêmes données via le Worker (`GET /sites`), qui applique **la même sélection de colonnes**.

01_process_las.R — LiDAR → bathymétrie

- Entrée : `las/LITTO3D_BaieSaintMalo_ZH.las` (LITTO3D, **Lambert-93 EPSG:2154**, ~ 206 M points, altitudes rapportées au **zéro hydrographique**).
- Le LAS 1.2 format 0 ne contient pas le champ `Classification` → on n'utilise pas `grid_terrain` (lidR) mais directement `terra::rasterize(mean Z)`.
- Pour chaque site (reprojeté en L93) : clip sur une bbox + buffer 300 m, **MNT à 0,5 m**, extraction `profMin` / `profMax` et d'un **transect Est → Ouest** (100 points), génération d'une **miniature PNG 256×256** dans `pwa/data/thumbs/<siteID>_thumb.png`.
- Sorties : `data/tiles/` (MNT intermédiaires), `data/bathy_sites.json` **et** `pwa/data/bathy_sites.json` (version allégée : `grid` conservée pour le transect libre côté client), miniatures directement dans `pwa/data/thumbs/`.
- Écrit aussi `profMin` / `profMax` dans `pwa/data/sites.geojson` (fusion par `siteID`).
- Options CLI : `--res`, `--n`, `--start` (tests partiels).

04_marees_fes.py — marées FES2022

- Décompresse les `.nc.xz` de `fes2022/` un par un (34 constituantes harmoniques), extrait amplitude/phase au point **Saint-Malo (48,637 N ; -2,025 E)**, écrit `data/constituantes_stmalo.json`.
- Calcule la marée de **aujourd'hui** à `DATE_END` (par défaut `2028-12-12`, surchargeable par la variable d'environnement `MAREES_DATE_END`) via `pyfes.evaluate_tide_from_constituents`, pas de 10 min.
- Détecte PM/BM, calcule les **coefficients** (référence marnage vive-eau Saint-Malo = 1366 cm pour coeff 120), heures locales `Europe/Paris`.
- Sorties : `data/marees.json` **et** `pwa/data/marees.json`.

05_courants_fes.py — courants de marée (hors build)

- Entrées : `currents/eastward_velocity.tar.xz` et `northward_velocity.tar.xz` (composantes U/V de l'atlas FES, constituantes M2, S2, N2, K1, O1 a minima).
- Pour chaque point d'une grille (~1/30°) sur la bbox baie de Saint-Malo, ajustement moindres carrés des constituantes **effectives** (amplitude + phase) sur une simulation.
- Sorties : `data/courants_grid.json`, `pwa/data/courants_grid.json`, `docs/data/courants_grid.json` (si `docs/` existe).
- Options : `--test`, `--no-pyfes`, `--force`.

03_generate_profile.R — utilitaire ponctuel

Extrait un profil topographique entre deux points depuis un MNT GeoTIFF (`data/tiles/*.tif`), avec ajustement optionnel de la marée. Utilisé pour du diagnostic / des tracés manuels, pas dans le pipeline.

Prérequis pipeline : R ≥ 4.3 avec `lidR`, `terra`, `sf`, `googlesheets4`, `jsonlite`, `future` ; Python (`.venv/`) avec `pyfes`, `netCDF4`, `numpy`.

B.5 La PWA — structure et chargement

- `pwa/index.html` — page unique. Contient : écran de connexion, bandeau offline, header, bandeau marées, conteneur carte, widget port, HUD navigation, panneau liste des sites, fiche site (3 onglets), et les modales Prévision, Marées, Météo, Retour d'expérience, CGU, Export marées, plus l'overlay du tutoriel.
- `pwa/manifest.json` — `display: standalone`, `theme_color #0c3e44`, `background_color #031f1b`, `lang fr`, icônes 192 / 512 (maskable).
- **Polices** : Google Fonts (Oswald + Signika) — chargées via CDN, donc **absentes hors-ligne** (fallback système).
- **Bibliothèques locales** (`pwa/libs/` , pour l'offline) : **Leaflet** (carte), **Turf.js** (`turf.bearing`, `turf.distance`).
- **Bibliothèque externe (CDN, non offline)** : `@openmeteo/weather-map-layer` (unpkg) — couches météo raster. Sans réseau, les overlays météo (température, précipitations) sont donc indisponibles ; le reste fonctionne.

Ordre de chargement des scripts (`index.html`)

```
leaflet.js → @openmeteo/weather-map-layer → turf.min.js
→ config.js → marees.js → mareesite.js → port.js → bathy.js → courants.js
→ carte.js → sites.js → prevision.js → navigation.js → meteo.js
→ biplongee.js → cgu.js → retourexperience.js → mareesexport.js
→ tutorial.js → auth.js → app.js
```

Puis : enregistrement du Service Worker, et au `DOMContentLoaded` : `Auth.init() => App.init()`.

Séquence d'initialisation (`app.js` → `_startApp`)

L'ordre reflète les **dépendances** :

```

horloge locale
1. Marees.init()          (bandeau immédiat – requis avant la carte)
2. Bathy.init()           (chargement silencieux du JSON)
3. Courants.init()        (doit précéder Carte : Carte lit Courants.isDisponible())
4. Carte.init()           (couches + contrôles)
5. Sites.init() → Carte.afficherSites()
6. Navigation.demarrerGPS()
7. Sites.initOnglets()
8. Prevision.init()
9. Port.init()
10. BiPlongee.init()
11. Cgu.init()
12. RetourExperience.init() (+ flush de la file d'attente)
13. MareesExport.init()
14. Tutorial.init()       (au 1er démarrage uniquement)
_bindEvents() + _monitorOnline()

```

Dépendances entre modules

```

config.js          (aucune dépendance – seul fichier à adapter pour un autre club)
marees.js          ← config
mareesite.js       ← config, marees
bathy.js           ← config
courants.js        ← Leaflet
port.js            ← config, marees
carte.js           ← config, Leaflet, courants (optionnel), OMWeatherMapLayer
sites.js           ← config, carte, marees, mareesite, bathy, meteo, courants
navigation.js      ← config, carte, turf
meteo.js           ← config
prevision.js       ← marees, mareesite, sites, port, courants, biplongee
biplongee.js       ← sites, bathy, marees, mareesite, port, prevision
retourexperience.js ← config, sites, marees, auth
mareesexport.js    ← marees, port, config
cgu.js / tutorial.js / auth.js (autonomes)
app.js            ← tous les précédents

```

B.6 Modules JavaScript, un par un

i Information

Convention : chaque module est une **IIFE** exposant un objet global (`const X = (() => { ... })()`). Les fonctions préfixées `_` sont privées.

config.js — configuration globale (CONFIG)

Le **seul** fichier à modifier pour adapter l'application à une autre zone / un autre club.

Clé	Contenu
CONFIG.DATA	bathy = chemin local ; sites et marees = URLs du Worker (.../sites , .../marees), résolues en bas de fichier
CONFIG.CARTE	centre [48.68, -2.02] , zoom 12, min 8, max 18
CONFIG.TILES	8 sources : osm , esriOcean (+ esriOceanRef), ignPlan (WMTS Géoportail), shom (COASTALMAPS Géoportail), openSeaMap , litto3d (WMS SHOM), ownWind / ownPrecip (OpenWeatherMap, nécessite une clé – legacy)
CONFIG.NAV	vitesseDefaut 6 kt, watchGPS true

Clé	Contenu
CONFIG.MAREES	MSL_SCALE 0.9822, MSL_OFFSET_M 6.5278 (recalage FES → ZH SHOM)
CONFIG.PORT	seuilZH 2.0 m ; bateaux = Maclow 1,3 / Cassiopée 1,1 / Neptune 0,7
CONFIG.RETOUR_EXPERIENCE	workerUrl (https://mf-wms-proxy.reg-gallon.workers.dev), listes etatMer (0-4), vent , courant ; bateaux résolu depuis PORT.bateaux
CONFIG.AUTH	workerUrl (même Worker)
CONFIG.METEO	Saint-Malo 48.65 / -2.02 , timeout 5000 ms
CONFIG.METEO_FRANCE	réservé (données chiffrées via proxy) — non utilisé actuellement
CONFIG.OWM	apiKey (renseigné par tokens.js , optionnel)
CONFIG.TYPE_SITE	récif 🪸 / épave 🚢 / roche 🪨 / défaut 📍 → classes CSS
CONFIG.TYPE_MOUILLAGE	fixe 🧱 / ancre ⚓ / gueuse 🟡
SW_CACHE_VERSION	'smpe-v9' — constante historique (le SW utilise sa propre VERSION)

app.js — orchestrateur (App)

Point d'entrée. `App.init()` lance la séquence ci-dessus. Callback `_onSiteSelectionne(feature)` : centre la carte, active l'overlay MNT, charge la météo du site, met le site comme destination du HUD, affiche le bouton « Naviguer ». `_bindEvents()` câble **tous** les événements d'interface (menus, filtres, recherche, ouverture/fermeture des modales, touche Échap). `_monitorOnline()` bascule le bandeau offline sur `navigator.onLine`. `App.ouvrirFiche(siteID)` est appelé depuis le HTML des popups Leaflet.

auth.js — authentication (Auth)

Compte individuel par **lien magique**. Détails en B.11. API : `Auth.init(onSuccess)` , `Auth.isAuthenticated()` , `Auth.getUser()` ({ email, nom }), `Auth.logout()` . Session dans `localStorage` (`smpe_auth_v2`), identifiant d'appareil dans `smpe_device_id` .

carte.js — carte Leaflet (Carte)

- `init()` : instancie la carte, ajoute les fonds (**SHOM Marine par défaut**, IGN Plan, ESRI Ocean, OpenStreetMap), les overlays (OpenSeaMap **actif par défaut**, Litto3D SHOM WMS, **Courants marée**, Température, **Vent** (barbules), Précipitations), le **contrôle des couches** (haut-droit), le **widget vent Open-Meteo** (bas-gauche) et le **widget filtre mouillage** (haut-droit, vrai contrôle Leaflet).
- Couches météo : adaptateur `OMWeatherMapLayer` + tuiles `map-tiles.open-meteo.com` (modèle `meteofrance_arome_france_hd`).
- Couche **vent = barbules** dessinées sur un canvas (`_WindBarbLayer`), requête batch Open-Meteo à chaque déplacement (cache 15 min par centre+zoom).
- Marqueurs : `L.divIcon` colorée par type + pastille d'état marée. Popup avec bouton « Voir la fiche ».
- API : `afficherSites` , `majEtatsMaree` , `filtrerMarqueurs` , `afficherGPS` , `centrerSurGPS` , `centrerSurSite` , `afficherLigneNav` / `supprimerLigneNav` , `toggleOverlayBathy(siteID)` , `setOverlayOpacity` / `getOverlayOpacity` , `getMap` .

sites.js — sites : liste + fiche (Sites)

Le module le plus riche.

- `init(cb)` : `fetch(CONFIG.DATA.sites)` (Worker), fusionne `profMin / profMax` depuis `Bathy`, affiche la liste, **rafraîchit les états marée toutes les 60 s**, et **recharge /sites** (avec `cache: 'reload'`) sur `visibilitychange`, `focus`, `online` et un poll de secours 5 min — throttle 10 s (`rafraichir()`).
- `filtrer(terme, type, prof, mouillage)` : filtres cumulables, répercutés sur les marqueurs. Le filtre profondeur utilise la **hauteur de marée actuelle**.
- `selectionner(siteID)` : ouvre la fiche + callback vers carte/navigation.
- Fiche (`_ouvrirFiche`) : remplit les 3 onglets, le bloc marée (`MaréeSite.rendreBloc`), le bloc courants (`Courants.renderBlocFiche`), les profondeurs dynamiques, la bathymétrie (miniature + `Bathy.dessiner`), gère le **mode transect libre** (2 clics sur la miniature → coordonnées Lambert-93 → `Bathy.dessinerTransectLibre`) et le slider d'opacité de l'overlay.
- API : `init`, `rafraichir`, `filtrer`, `selectionner`, `fermerFiche`, `getGeojson`, `getSiteActif`, `getSiteById`, `initOnglets`.

marees.js — marées (Marees)

- `init()` : `fetch(CONFIG.DATA.marees)` (Worker), **normalise** chaque entrée : `*_haut (m ZH) = *_hcm/100 × MSL_SCALE + MSL_OFFSET_M`.
- Bandeau : hauteur actuelle (**interpolation sinusoïdale** entre extrêmes), sens ▲/▼, PM/BM triées, coefficient, indicateur d'étalement $PM \leq 2 \text{ h}$.
- `ouvrirModal()` : courbe 4 jours (canvas) + tableau PM/BM.
- API : `getData`, `getAujourd`, `getEntreePourDate(date)`, `getHauteurAt(date)`, `getHauteurActuelle()`, `getExtremaJour()`, `getEtalementProche(date)`.

mareesite.js — moteur de plongeabilité (MaréeSite)

Interprète le champ `maree` du site (codes, voir B.13) et le champ `tpsEtale` (durée de la fenêtre) au regard de l'entrée `marees.json` du jour :

- Type d'eau du jour : **ME** si coeff ≤ 70 , **VE** sinon.
- Pour chaque code, calcule la (les) **fenêtre(s)** [début, fin] autour de l'étalement correspondante (`H` = centrée ; `A` = commence decal min avant ; `R` = commence decal min après).
- Statut : **vert** (dans une fenêtre), **orange** (fenêtre < 2 h), **rouge** (passée / type d'eau incompatible), **gris** (pas de code ou données de marée manquantes).
- API : `calculerEtat(props, entree, [now])`, `calculerTous(geojson, entree)` → `Map<siteID, etat>`, `rendreBloc(props, entree)` → HTML, `getFenetres(props, entree)` (utilisé par `BiPlongee`, indépendant de l'heure courante).

bathy.js — bathymétrie LiDAR (Bathy)

- `init()` : `fetch(CONFIG.DATA.bathy)` → `Map<siteID, entrée>`.
- `get(siteID)`, `isLoading()`.
- `dessiner(canvas, siteID, hMaree)` : profil Est → Ouest, ligne de surface à la marée, dégradé de fond, annotation de profondeur, axe des distances.
- `dessinerTransectLibre(canvas, siteID, ptA, ptB, hMaree)` : **interpolation bilinéaire** dans la grille 0,5 m (`entrée.grid`) le long du segment A → B (80 points), puis même rendu.

navigation.js — GPS & cap (Navigation)

- `demarrerGPS()` : `watchPosition({ enableHighAccuracy: true, timeout: 15000, maximumAge: 5000 })`, met à jour le marqueur et le HUD (DMS, cap `heading`, vitesse en nœuds, distance au site via `turf.distance`).
- `naviguerVers(feature)` : trace la ligne de cap, calcule cap (`turf.bearing`) et ETA (`CONFIG.NAV.vitesseDefaut`).
- API : `demarrerGPS` / `arreterGPS`, `centrerSurMoi`, `naviguerVers`, `arreter`, `setSiteDestination(feature)`, `getPosition`, `isActif`.

meteo.js — météo marine (Meteo)

- Trois sources **Open-Meteo** (sans clé), agrégées via `Promise.allSettled` : AROME PI (`current`), AROME HD (`hourly`, 2 j), API Marine (`wave_*`).
- Cache mémoire **30 min**. Décodage de ~20 codes météo **WMO** (emoji).
- `ouvrirModal()` : conditions + houle + prévision vent 24 h.
- `chargerPourSite(lat, lon)` : bloc compact dans l'onglet Conditions.

courants.js — courants de marée (Courants)

- `init()` : `fetch('data/courants_grid.json')` ; `_grid = null` si absent (couche affichée avec ⚠ et popup d'aide).
- **Synthèse harmonique côté client** : $u(t) = \sum A_n \cdot \cos(\omega_n \cdot \Delta t + \phi_n)$ (U est, V nord), Δt en heures depuis `meta.t_ref`.
- `creerCouche()` : `L.Layer` canvas dessinant des **flèches** (couleur/longueur $\propto$ vitesse) aux points de grille visibles.
- `ajouterControle(map)` : panneau temps réel / animation +24 h / sélecteur date-heure + légende + avertissement de résolution.
- API : `getVitesseSite(lat, lon, date)` $\rightarrow$ { `u`, `v`, `vitesse` (cm/s), `dirFrom`, `dirTo` }, `renderBlocFiche(lat, lon)`, `previsionJournee(lat, lon, date)`, `setTemps` / `setTempsReel` / `setAnimation`, `isDisponible()`.

port.js — seuil du port (Port)

- Reconstitue la **courbe de hauteur** de la journée (pas 5 min, interpolation sinusoïdale) à partir d'une entrée `marees.json`.
- `getEtatActuel(hauteur, entree, [now])` $\rightarrow$ par bateau : `peut`, `prochainAcces`, `prochainBlocage`.
- `getFenetresJour(entree)` $\rightarrow$ par bateau : `hMin`, `plages` `bloque[]`.
- `updateWidgetCarte(...)` (widget flottant, rafraîchi 60 s), `renderPrevision(entree, hauteur, targetDate)` (bloc de la modale Prévission).

prevision.js — prévision de plongéabilité (Prevision)

- `ouvrir()` : pré-remplit date/heure, `_calculer()`.
- `_calculer()` : mode **1 plongée** $\rightarrow$ hauteur + mini-courbe + bloc Port + `MaréeSite.calculerEtat` pour tous les sites, classés vert/orange/rouge/gris, filtre profondeur réelle, badge courant par site. Mode **2 tanks** $\rightarrow$ délègue à `BiPlongee.afficher(dateStr, 'prev-bi-resultats')`.
- Gère la bascule « sites prioritaires / tous les sites » (colonne `prioritePrevision`), partagée avec le mode bi-journée (`getFeaturesActives`).

biplongee.js — planificateur bi-journée (BiPlongee)

- Constantes : cale du Naye 48.6384 / -2.0235 , **15 kt**, coefficient de navigation **1,35**, plongée **45 min**, intervalle de surface réglable (definirIntervalleSurface , défaut 30–180 min), marge de fin d'étale 5 min.
- afficher(dateStr, containerId) : pré-calcul O(n) par site (transit depuis le port, fenêtres), puis **balayage O(n²)** des paires × créneaux de départ (7 h → 22 h, pas 5 min) par lots de 80 (barre de progression). Rejette : hors fenêtre d'étale, surface hors bornes, **profil inversé**. Retient la **fenêtre de départ** [deptMin, deptMax], l'heure de retour au port et son état (via Port.getFenetresJour). Recherche par nom dans les résultats.

retourexperience.js — retour d'expérience (RetourExperience)

- ouvrir([siteID]) : formulaire (site + date obligatoires). Recalcule en direct hauteur d'eau, étale proche, coefficient (Marees).
- Soumission → POST {workerUrl}/retour-experience . Échec réseau → **file d'attente** localStorage (smpe_retour_queue) + pastille sur le bouton  ; flushQueue() au chargement et sur online .

mareesexport.js — export CSV (MareesExport)

Génère un CSV (; , BOM UTF-8) sur une période : par PM/BM → date, heure, type, coefficient, hauteur ; par jour → créneaux de blocage des 3 bateaux (Port.getFenetresJour).

cgu.js — modale CGU (Cgu)

open() / close() . Ouverte depuis l'écran de connexion et le bouton Contenu (CGU v1.1, mai 2026) directement dans index.html .

tutorial.js — tutoriel guidé (Tutorial)

9 étapes « spotlight » (masque troué + carte de texte positionnée intelligemment). État dans localStorage (smpe_tutorial_done). relancer() depuis ... →  Tutoriel .

B.7 Widgets et éléments d'interface

Élément	Emplacement	Rôle
Header	haut	Menu, horloge, GPS, Prévision, Retour d'expérience (+ pastille), Marées, Météo, menu ...
Bandeau marées	sous le header	Hauteur live, PM/BM, coefficient, étale
Bandeau offline	tout en haut	Visible quand navigator.onLine est faux
Contrôle des couches	carte, haut-droit	Fonds + overlays (OpenSeaMap, Litto3D, Courants, Température, Vent, Précipitations)
Widget filtre mouillage ()	carte, haut-droit	Filtre Fixe / Ancre / Gueuse
Widget vent	carte, bas-gauche	Vent + rafales à Saint-Malo (Open-Meteo, refresh 15 min)
HUD navigation	carte, bas-gauche	Position DMS, cap, vitesse, distance au site, bouton stop

Élément	Emplacement	Rôle
Widget port (🚢)	carte, bas-droit	État du seuil par bateau, prochain accès / blocage
Panneau courants	carte, bas-droit	Temps réel / animation / date-heure + légende
Onglet « 👁 Widgets »	carte, bord gauche	Masque/affiche HUD + port + vent d'un coup (classe <code>widgets-masques</code> sur <code><body></code>)
Overlay MNT	carte	Miniature du fond du site sélectionné, opacité réglable
Modales	plein écran	Prévision, Marées, Météo, Retour d'expérience, CGU, Export marées

B.8 Service Worker et cache hors-ligne

`pwa/sw.js` — constante `VERSION` (actuellement `'v59'`).

- Caches : `smpe-static-<VERSION>` (application + données locales), `smpe-dynamic-<VERSION>` (tuiles, **300 entrées max**, purge LRU).
- `install` : met en cache `ASSETS_STATIQUES` (index, manifest, CSS, tous les JS, `data/bathy_sites.json` , `data/courants_grid.json` , Leaflet, Turf, icônes) ; tolérant aux erreurs unitaires ; `skipWaiting()` .
- `activate` : supprime les caches inconnus, `clients.claim()` , notifie les clients (`SW_UPDATED`).
- `fetch` — stratégies :

Ressource	Stratégie
<code>open-meteo.com</code> , Worker <code>/sites</code> et <code>/marees</code>	Network First (fallback cache, puis <code>503 {error:"offline"}</code>)
Tuiles <code>tile.openstreetmap.org</code> , <code>tiles.openseamap.org</code> , <code>wxs.ign.fr</code>	Cache First + revalidation réseau (timeout 8 s)
Tout le reste (assets de l'app)	Cache First strict (fallback offline typé JSON/image/HTML)

- `index.html` détecte un nouveau SW (`updatefound` / `reg.waiting`), lui envoie `SKIP_WAITING` , et recharge la page sur `controllerchange` .

⚠ Attention

Règle : toute modification de `ASSETS_STATIQUES` ou d'un fichier statique listé (ajout d'un module JS, etc.) **doit s'accompagner d'un bump de `VERSION`** , sinon les appareils déjà installés ne rechargent pas le cache.

B.9 Le Worker Cloudflare

`cloudflare-worker/mf-wms-proxy.js` — déployé via **Wrangler** (`wrangler.toml` : nom `mf-wms-proxy` , binding KV `MAREES_KV`). URL : `https://mf-wms-proxy.reg-gallon.workers.dev` .

CORS : origines autorisées `https://gallonr.github.io` , `http://localhost` , `http://127.0.0.1` , `null` (file://).

Route	Méthode	Rôle
/sites	GET	Lit l'onglet <code>site</code> via <code>bdd.gs</code> , transforme en GeoJSON (mêmes colonnes que <code>02_process_bdd.R</code>). Cache KV en <i>stale-while-revalidate</i> : réponse immédiate depuis la copie KV (~50 ms), rafraîchie en tâche de fond si > 60 s ; copie de secours conservée 24 h ; <code>?nocache=1</code> (alias <code>?fresh=1</code>) force un appel synchrone et remonte l'erreur réelle.
/marees	GET	Renvoie la valeur de la clé <code>marees</code> du KV (<code>marees.json</code> publié par <code>sync_docs.sh</code>).
/auth/request-link, /auth/verify	POST	Relaie vers <code>auth.gs</code> (ajoute le secret partagé).
/retour-experience	POST	Relaie vers <code>retour-experience.gs</code> (ajoute le secret partagé).
/paarome, /aromepi	GET	Proxy WMS Météo-France : injecte le token serveur (<code>MF_TOKEN_PAAROME</code> / <code>MF_TOKEN_AROMEPI</code>), recopie les paramètres WMS, renvoie l'image (cache 15 min).

Les tokens et secrets ne sont **jamais** exposés côté client. Le KV `MAREES_KV` stocke deux entrées : `marees` (JSON marées) et `sites_geojson_cache` (copie GeoJSON + horodatage en métadonnée).

B.10 Les scripts Google Apps Script

Trois Web Apps déployées « Exécuter en tant que : Moi / Accès : Tout le monde », appelées **uniquement par le Worker** (secret partagé dans le corps de la requête). Elles pointent toutes vers le **même Google Sheet** (onglets distincts).

Script	Onglet	Fonction
<code>bdd.gs</code>	<code>site</code>	<code>doPost</code> : vérifie <code>BDD_APPSCRIPT_SECRET</code> , renvoie toutes les lignes (<code>{ ok, rows }</code>). <code>doGet</code> : ping de diagnostic.
<code>auth.gs</code>	<code>utilisateurs (email nom actif)</code>	<code>request-link</code> : vérifie le membre actif, construit un token signé HMAC lié au <code>device_id</code> , l'envoie par email (<code>MailApp</code>). <code>verify</code> : vérifie signature + expiration + <code>device_id</code> + statut <code>actif</code> .
<code>retour-experience.gs</code>	<code>retours_plongee</code>	<code>doPost</code> : vérifie <code>APPSCRIPT_SECRET</code> , <code>appendRow</code> selon l'ordre fixe <code>HEADERS</code> .

Propriétés de script requises : `bdd.gs` → `BDD_APPSCRIPT_SECRET`, `SHEET_ID` ; `auth.gs` → `AUTH_APPSCRIPT_SECRET`, `AUTH_TOKEN_SECRET` (clé HMAC **distincte**), `SHEET_ID`, `PWA_URL` ; `retour-experience.gs` → `APPSCRIPT_SECRET`, `SHEET_ID`.

B.11 Authentification

Modèle « **lien magique** » (remplace l'ancien login partagé) — cf. `specs/2026-08-25-auth-utilisateur-token-design.md`.

1. La PWA génère et stocke un `device_id` (`crypto.randomUUID()`, `localStorage: smpe_device_id`), jamais transmis par email.

2. `POST /auth/request-link { email, device_id }` → `auth.gs` vérifie que l'email est un membre **actif**, construit `token = base64url(JSON{email, exp, device_id}) + "." + HMAC_SHA256(...)` (validité **90 jours**), envoie le lien `PWA_URL?token=...` **et** le token brut (à coller) par email.
3. Au retour (`?token=` dans l'URL, ou collage manuel), la PWA appelle `POST /auth/verify { token, device_id }`. `auth.gs` recalcule le HMAC, vérifie l'expiration, **exige que le `device_id` du payload signé == celui présenté** (un token copié sur un autre appareil est rejeté), et que le compte est toujours **actif**.
4. Session : `localStorage: smpe_auth_v2 = { email, nom, token, ts }`, durée **90 j glissante**. `Auth.init` déverrouille immédiatement si la session est valide, puis **revalide en arrière-plan** dès qu'il y a du réseau (jamais bloquant en mer). Une réponse serveur `ok:false` explicite (compte désactivé) force la déconnexion avec le message « accès révoqué » ; une simple erreur réseau ne déconnecte jamais.

Révoquer un accès : passer `actif` à `FAUX` sur la ligne du membre (onglet `utilisateurs`). Effet à la prochaine revalidation en ligne de l'appareil.

B.12 Formats de données

`sites.geojson` / réponse `GET /sites`

FeatureCollection de `Point` (WGS84). Propriétés (colonnes de l'onglet `site`, miroir de `cols_voulues` dans `02_process_bdd.R` et `SITE_COLUMNS` dans le Worker) :

Champ	Type	Description
<code>siteID</code>	string	Identifiant unique
<code>siteNom</code>	string	Nom affiché
<code>latitude</code> , <code>longitude</code>	number	WGS84 (aussi dans <code>geometry</code>)
<code>typeSite</code>	string	<code>récif / épave / roche</code>
<code>accessibilite</code>	string	Texte libre
<code>typePlongee</code>	string	Texte libre
<code>niveauPlongee</code>	string	Niveau requis
<code>accesVent</code>	string	Orientations de vent à éviter
<code>houle</code>	string	Sensibilité à la houle
<code>mouillage</code>	string	Vocabulaire : <code>fixe / ancre / gueuse / vide</code>
<code>maree</code>	string	Code(s) de plongéabilité — voir B.13
<code>tpsEtale</code>	string	Durée de la fenêtre de plongée (<code>2h15</code> , <code>1h(VE)/1h30(ME)</code> , ...)
<code>commentaire</code>	string	Commentaire du responsable technique
<code>photoSite</code>	string	Référence photo (non affichée pour l'instant)
<code>prioritePrevision</code>	bool	Site mis en avant dans la Prévision (<code>VRAI</code> / <code>FALSE</code>)
<code>profMin</code> , <code>profMax</code>	number	Ajoutés côté client depuis <code>bathy_sites.json</code> (cote sous ZH, valeur positive = profondeur)

marees.json / réponse GET /marees

Objet indexé par date ISO "YYYY-MM-DD", une entrée par jour, de aujourd'hui à `DATE_END`. Champs par extrême (`PM1`, `BM1`, `PM2`, `BM2`, parfois `BM3`) :

```
"2026-09-01": {
  "PM1_h": "07:55", "PM1_coeff": 98, "PM1_hcm": 620,
  "BM1_h": "01:30", "BM1_hcm": -640,
  "PM2_h": "20:15", "PM2_coeff": 96, "PM2_hcm": 610,
  "BM2_h": "14:10", "BM2_hcm": -630
}
```

`*_hcm` = hauteur en **cm par rapport au MSL FES2022** (peut être négative). Le client ajoute `*_haut` (m au-dessus du ZH) via la calibration [B.14](#). Seules les PM portent un `*_coeff`.

bathy_sites.json

Tableau d'objets, un par site couvert par le LiDAR :

```
{
  "siteID": "...",
  "profMin": 2.1, "profMax": 18.5,
  "transect": { "dist_m": [0, ...], "z_m": [-2.1, ...] },
  "grid": {
    "ncol": ..., "nrow": ..., "res": 0.5,
    "xmin": ..., "ymin": ..., // Lambert-93 (EPSG:2154)
    "z": [ ... ], // grille aplatie, -9999 = nodata
    "bounds_wgs84": { "south": ..., "west": ..., "north": ..., "east": ... }
  }
}
```

`transect.z_m` : altitudes (négatives sous le zéro). `grid` : sert au **transect libre** (interpolation bilinéaire côté client) et à l'**overlay carte** (`bounds_wgs84`).

courants_grid.json

```
{
  "meta": {
    "t_ref": "2026-01-01T00:00:00Z",
    "n_points": ..., "res_deg": 0.0333,
    "constituants": [ "M2", "S2", "N2", "K1", "O1", ... ],
    "omega_deg_h": { "M2": 28.9841042, ... }
  },
  "points": [
    { "lat": ..., "lon": ...,
      "u": [amp0, phi0, amp1, phi1, ...], // Est, cm/s & degrés
      "v": [amp0, phi0, ...] } // Nord
    ]
}
```

Synthèse harmonique côté client : $val(t) = \sum amp_k \cdot \cos(\omega_k \cdot \Delta t + \phi_k)$, Δt en heures depuis `t_ref`.

Miniatures

`pwa/data/thumbs/<siteID>_thumb.png` — 256×256, vue de dessus du MNT, référencée par convention de nom (pas par le GeoJSON).

B.13 Les codes marée

Champ `maree` d'un site : un ou plusieurs codes séparés par `/`.

```
PM ME _ R 15'
| | | |
| | | | décalage : minutes (15') ou heures (2h30)
| | | | R = Retard (après l'été) · A = Avance (avant) · H = à l'été
| | | | ME = Morte-Eau (coeff ≤ 70) · VE = Vive-Eau (coeff > 70)
| | | | PM = Pleine Mer · BM = Basse Mer
```

Exemples : `PMME_R15'` (PM morte-eau, 15 min après l'été) · `BMVE_A2h30` (BM vive-eau, 2h30 avant l'été) · `PMVE_H` (PM vive-eau, à l'été) · `PMME_R15' / BMVE_A2h30` (l'un **ou** l'autre selon le jour).

Champ `tpsEtale` (durée de la fenêtre autour du moment défini par le code) : `2h15`, plage `1h-1h30` (→ moyenne), ou distinct par type d'eau `1h(VE)/1h30(ME)` (ordre libre). Défaut si absent : 1 h.

Calcul de la fenêtre [`début`, `fin`] (`mareesite.js`) :

dir	Fenêtre
H	[été - tpsEtale/2 ; été + tpsEtale/2]
A	[été - décal ; été - décal + tpsEtale]
R	[été + décal ; été + décal + tpsEtale]

Le statut compare l'heure courante (ou l'heure simulée) à ces fenêtres : **vert** dedans, **orange** si la prochaine est < 2 h, **rouge** sinon, **gris** si code absent / non reconnu ou horaires de marée manquants.

B.14 Calibration des marées FES2022

FES2022 surestime légèrement le marnage à Saint-Malo. Une régression linéaire (OLS) sur **27 points de l'annuaire SHOM** (17-23/04/2026, PM + BM) donne :

```
h_ZH (m) = hcm/100 × MSL_SCALE + MSL_OFFSET_M
          = hcm/100 × 0.9822      + 6.5278
```

RMS des résidus ≈ **0,16 m** ; biais BM ≈ -0,01 m, biais PM ≈ +0,02 m. Ces deux constantes sont dans `CONFIG.MAREES` (`config.js`) et appliquées à l'`init` de `marees.js`. Les coefficients, eux, sont calculés côté Python (`MARNAGE_VE_REF_CM = 1366` pour le coefficient 120).

B.15 Déploiement

PWA → GitHub Pages

Le dossier `docs/` est la copie publiée (<https://gallonr.github.io/diveSMPE/>). **Ne jamais l'éditer directement.** Utiliser :

```
./sync_docs.sh "feat: ..."
```

Ce script :

- copie `pwa/js/*.js`, `pwa/css/style.css`, `pwa/sw.js`, `pwa/manifest.json` vers `docs/` ;

2. copie `pwa/index.html` → `docs/index.html` en **corrigéant** le lien du guide utilisateur (`https://gallonr.github.io/diveSMPE/guide-utilisateur.html` → `guide-utilisateur.html`);
3. copie `pwa/data/bathy_sites.json`, `pwa/data/courants_grid.json` et synchronise `pwa/data/thumbs/` (`rsync --delete`);
4. **publie** `pwa/data/marees.json` dans le KV Cloudflare (`wrangler kv key put "marees" --binding=MAREES_KV --path=... --remote`, via `wrangler global` ou `npx wrangler`);
5. `git add -A && git commit && git push`.

i Information

`sites.geojson` et `marees.json` **ne sont plus** copiés dans `docs/` : la PWA les obtient du Worker.

Worker Cloudflare

Déploiement séparé, via Wrangler, depuis `cloudflare-worker/` (`wrangler deploy`). Les secrets se configurent dans les variables du Worker (dashboard Cloudflare ou `wrangler secret put`) — voir C.7.

Contrainte HTTPS

Le Service Worker n'est servi qu'en **HTTPS ou localhost**. GitHub Pages est en HTTPS ; en dev local, `npx http-server` sur `localhost` suffit.

Partie C — Maintenance & exploitation

C.1 Cycle de mise à jour complet

- | | |
|---|-----------------------------------|
| 1. Modifier le Google Sheet (onglet "site") | ← ajout/correction de site |
| 2. (au centre) Rscript r/build_all.R | ← régénère data/ + pwa/data/ |
| 3. (si besoin) python r/05_courants_fes.py | ← si atlas courants mis à jour |
| 4. ./sync_docs.sh "message" | ← publie docs/ + KV marées + push |
| 5. Chaque appareil, en ligne → ouvrir l'application | ← le SW se met à jour seul |

Cas particuliers :

- **Modification de site uniquement** (texte, code marée, coordonnées) : le Worker `GET /sites` relit le Sheet automatiquement (cache 60 s). Inutile de rebuild ou de resync pour que l'appareil voie le changement (au prochain passage au premier plan, s'il a du réseau). Un rebuild reste nécessaire si la **bathymétrie** d'un nouveau site doit être calculée.
- **Nouvelle version de code PWA** : étapes 4 + 5, avec **bump de VERSION** dans `pwa/sw.js` (voir C.4).

C.2 Ajouter ou modifier un site

1. Ouvrir le **Google Sheet BDD**, onglet `site`.
2. Renseigner au minimum `siteID`, `siteNom`, `latitude`, `longitude`, `typeSite`. Pour la plongéabilité : `maree` (codes, B.13) et `tpsEtale`. Pour le mouillage : `fixe` / `ancree` / `gueuse` (ou vide).
3. `latitude` / `longitude` en degrés décimaux WGS84, dans l'emprise baie de Saint-Malo (sinon `02_process_bdd.R` échoue au contrôle d'emprise).
4. Pour la **bathymétrie** : le site doit être dans l'emprise du LiDAR ; relancer `Rscript r/build_all.R` (étape LiDAR) puis `./sync_docs.sh`.
5. Sans bathymétrie : le site apparaît quand même (le Worker relit le Sheet), sans l'onglet Bathymétrie complet.

C.3 Régénérer les données

```
# depuis la racine du projet
Rscript r/build_all.R          # sites + bathy + marées + validation
python r/05_courants_fes.py    # courants (si atlas mis à jour) – hors build
```

Prérequis : R ≥ 4.3 (`lidR`, `terra`, `sf`, `googlesheets4`, `jsonlite`, `future`), Python `.venv/` (`pyfes`, `netCDF4`, `numpy`), fichiers volumineux en place (`las/`, `fes2022/`, `currents/`), `r/config_local.R` renseigné (`GOOGLE_SHEET_BDD_ID`).

`build_all.R` échoue si : fichier manquant, < 50 sites, < 360 jours de marées, < 40 sites bathy.

C.4 Publier une nouvelle version de la PWA

1. Modifier les fichiers sous `pwa/` (jamais `docs/`).
2. Si `ASSETS_STATIQUES` change (nouveau module JS) ou si un fichier statique listé change : **incrémenter** **VERSION** dans `pwa/sw.js` (`'v59'` → `'v60'`). Ajouter la ligne `BASE + 'js/<nouveau-module>.js'` dans

ASSETS_STATIQUES , et la balise `<script>` correspondante dans `pwa/index.html` (au bon rang de dépendance).

3. `./sync_docs.sh "feat: ..."`.
4. Vérifier sur <https://gallonr.github.io/diveSMPE/> (ouvrir la console : la nouvelle `VERSION` doit apparaître, la page se recharge une fois).

C.5 Gérer les utilisateurs

- **Ajouter** : nouvelle ligne dans l'onglet `utilisateurs` du Sheet (`email` | `nom` | `actif`), `actif` = `VRAI` . La personne peut alors demander son lien depuis l'écran de connexion.
- **Révoquer** : passer `actif` à `FAUX` . Effet à la prochaine revalidation en ligne de l'appareil concerné.
- Un utilisateur = potentiellement plusieurs appareils (chacun avec son `device_id` et sa demande de lien).

C.6 Prolonger la période des marées

Les marées vont d'aujourd'hui à `DATE_END` (`2028-12-12` par défaut dans `r/04_marees_fes.py`). Pour repousser l'échéance :

```
MAREES_DATE_END=2030-12-31 Rscript r/build_all.R
./sync_docs.sh "chore: marées jusqu'à fin 2030"
```

(ou modifier la constante `DATE_END` dans le script). La validation exige ≥ 360 jours.

C.7 Secrets et variables d'environnement

Emplacement	Nom	Rôle
<code>r/config_local.R</code> (gitignoré)	<code>GOOGLE_SHEET_BDD_ID</code>	ID du Google Sheet BDD
Worker Cloudflare (secrets)	<code>BDD_APPSCRIPT_URL</code> , <code>BDD_APPSCRIPT_SECRET</code>	Route <code>/sites</code> → <code>bdd.gs</code>
	<code>AUTH_APPSCRIPT_URL</code> , <code>AUTH_APPSCRIPT_SECRET</code>	Routes <code>/auth/*</code> → <code>auth.gs</code>
	<code>APPSCRIPT_URL</code> , <code>APPSCRIPT_SECRET</code>	Route <code>/retour-experience</code> → <code>retour-experience.gs</code>
	<code>MF_TOKEN_PAAROME</code> , <code>MF_TOKEN_AROMEPI</code>	Proxy WMS Météo-France
Worker <code>wrangler.toml</code>	binding <code>MAREES_KV</code>	Clés <code>marees</code> + <code>sites_geojson_cache</code>
Apps Script <code>bdd.gs</code>	<code>BDD_APPSCRIPT_SECRET</code> , <code>SHEET_ID</code>	
Apps Script <code>auth.gs</code>	<code>AUTH_APPSCRIPT_SECRET</code> , <code>AUTH_TOKEN_SECRET</code> , <code>SHEET_ID</code> , <code>PWA_URL</code>	<code>AUTH_TOKEN_SECRET</code> = clé HMAC, distincte du secret d'appel

Emplacement	Nom	Rôle
Apps Script <code>retour-experience.gs</code>	<code>APPSCRIPT_SECRET</code> , <code>SHEET_ID</code>	
<code>pwa/js/tokens.js</code> (gitignoré)	<code>CONFIG.OWM.apiKey</code>	Clé OpenWeatherMap (overlays tuiles — optionnel, non copié vers <code>docs/</code>)
<code>pwa/js/secrets.js</code> (gitignoré)	—	Template <code>secrets.js.example</code> ; clés éventuelles (Météo-France côté client) — préférer le passage par le Worker

Générer un secret : `openssl rand -hex 32` . Les secrets d'appel Apps Script doivent être **identiques** des deux côtés (Worker ↔ Apps Script).

C.8 Dépannage

Symptôme	Piste
La liste des sites reste vide / « Erreur de chargement »	Tester <code>GET https://mf-wms-proxy.reg-gallon.workers.dev/sites?nocache=1</code> : renvoie l'erreur réelle (Apps Script lent/HS, secret, <code>SHEET_ID</code> , onglet <code>site</code>).
Un site modifié dans le Sheet n'apparaît pas	Attendre ≤ 60 s + repasser l'app au premier plan (recharge <code>/sites</code>). Sinon ? <code>nocache=1</code> .
Marées « N/A »	KV vide : relancer <code>./sync_docs.sh</code> (publie <code>marees.json</code> dans le KV) ; vérifier <code>wrangler</code> disponible.
Un appareil garde une vieille version	<code>VERSION</code> non bumpée dans <code>pwa/sw.js</code> , ou appareil jamais rouvert en ligne. Forcer : vider les données du site dans le navigateur, ou désinstaller / réinstaller la PWA.
Couche « Courants » avec ⚠	<code>courants_grid.json</code> absent : <code>python r/05_courants_fes.py</code> puis <code>./sync_docs.sh</code> .
Overlays météo (vent/ température) vides	Réseau requis et CDN unpkg accessible (<code>@openmeteo/weather-map-layer</code>) — indisponibles hors-ligne.
Lien de connexion qui ne marche pas	Utiliser le token à coller de l'email. Vérifier que la demande a été faite depuis le même appareil (le token est lié au <code>device_id</code>).
<code>build_all.R</code> échoue au contrôle d'emprise	<code>latitude / longitude</code> d'un site hors baie de Saint-Malo, ou colonnes inversées.
<code>build_all.R : python3 introuvable</code>	Activer le <code>.venv/</code> ou ajouter Python au PATH.
Miniature bathy absente sur un site	Site hors emprise LiDAR LITTO3D (~16 sites concernés). Comportement normal.

Annexes

Glossaire

Terme	Définition
PWA	Progressive Web App — site web installable, fonctionnant hors-ligne
Service Worker	Script navigateur qui intercepte les requêtes et gère le cache offline
ZH	Zéro hydrographique (SHOM) — référence des sondes des cartes marines
MSL	Mean Sea Level — niveau moyen de la mer (référence de FES2022)
FES2022	Modèle global de marée par constituantes harmoniques (CNES/LEGOS)
LITTO3D / LiDAR	Relevé altimétrique haute résolution du littoral (IGN/SHOM)
MNT	Modèle Numérique de Terrain — grille d'altitudes
Étale	Moment de renversement du courant, autour de la PM ou de la BM
ME / VE	Morte-Eau (coeff ≤ 70) / Vive-Eau (coeff > 70)
Transect	Coupe verticale du fond le long d'une ligne
KV	Key-Value store de Cloudflare Workers
HMAC	Signature cryptographique à clé secrète (intégrité du token)
Lambert-93	Projection cartographique française (EPSG:2154)
WGS84	Système de coordonnées géographiques mondial (EPSG:4326)

Documents de conception (specs/)

Fichier	Sujet
2026-07-28-type-mouillage-design.md	Vocabulaire contrôlé mouillage
2026-07-28-retour-experience-design.md + ...-implementation-plan.md	Retour d'expérience post-plongée
2026-08-25-auth-utilisateur-token-design.md + ...-implementation-plan.md	Authentification par lien magique
2026-08-31-live-worker-data-design.md + ...-implementation-plan.md	Données sites / mares servies en direct par le Worker

Autres documents du dépôt

Fichier	Contenu
<code>README.md</code>	Présentation courte du projet
<code>CLAUDE.md</code>	Instructions pour l'assistant Claude Code
<code>pwa/technicalDescription.md</code>	Description technique du dossier <code>pwa/</code> (antérieure à cette notice)
<code>docs/guide-utilisateur.md / .html</code>	Guide utilisateur publié (lien depuis le menu <code>...</code>)
<code>AUDIT_RISQUES.md</code>	Analyse de risques
<code>bdd/README.md</code>	Mise en place du Google Sheet BDD
<code>google-apps-script/README.md</code> , <code>README-auth.md</code>	Déploiement des Web Apps

Sources de données et attributions

- **Marées** : FES2022 — CNES/LEGOS/AVISO ; recalage annuaire SHOM Saint-Malo.
- **Bathymétrie** : LiDAR LITTO3D — IGN / SHOM.
- **Courants** : atlas de courants FES.
- **Fonds de carte** : SHOM/IGN (Géoportail COASTALMAPS), IGN Plan V2, ESRI Ocean, OpenStreetMap, OpenSeaMap.
- **Météo** : Météo-France (AROME, MFWAM) via Open-Meteo ; proxy WMS via le Worker.

Club SMPE — Saint-Malo Plongée Emeraude — Baie de Saint-Malo. Application et code : Régis GALLON (voir CGU dans l'application).